



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Контрольная работа № 2

Темы:

3.1 Работа с триггерами. Часть 1

3.2 Работа с триггерами. Часть 2

Выполнил студент 2 курса группы
ЭФБО-04-24 Валекжанин В.С.

Проверил доцент Бочаров М.И.

Москва
2026 г.

3.1 Работа с триггерами. Часть 1

Цель: изучить работу с триггерами.

3.1.1 Вывод \df

List of functions				
Schema	Name	Result data type	Argument data types	Type
public	check_price_difference_valekzhanin	trigger		func
public	check_sale_date_valekzhanin	trigger		func
public	check_structure_area_valekzhanin	trigger		func
public	log_sale_action_valekzhanin	trigger		func
public	prevent_duplicate_sale_valekzhanin	trigger		func
(5 rows)				

Задание 1

1. Создать триггер, который при продаже будет выводить предупреждающее сообщение при разнице заявленной и продажной стоимости объекта недвижимости более чем на 20%.

```
CREATE OR REPLACE FUNCTION check_price_difference_valekzhanin()
RETURNS TRIGGER AS $$
DECLARE
    v_listed_price NUMERIC;
    v_sale_price NUMERIC;
    v_difference_percent NUMERIC;
    v_threshold_percent CONSTANT NUMERIC := 20.0;
BEGIN
    SELECT price INTO v_listed_price
    FROM property
    WHERE id = NEW.property_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;
    END IF;

    IF v_listed_price IS NULL OR NEW.price IS NULL THEN
        RAISE EXCEPTION 'Цена не может быть NULL';
    END IF;

    IF v_listed_price <= 0 OR NEW.price <= 0 THEN
        RAISE EXCEPTION 'Цена должна быть положительным числом';
    END IF;

    v_sale_price := NEW.price;
    v_difference_percent := ABS((v_sale_price - v_listed_price) / v_listed_price * 100);

    IF v_difference_percent > v_threshold_percent THEN
        RAISE WARNING 'ВНИМАНИЕ: Разница между заявленной (%) и продажной (%)
стоимостью составляет % (порог: % )',
            TO_CHAR(v_listed_price, 'FM999999990.00'),
            TO_CHAR(v_sale_price, 'FM999999990.00'),
            TO_CHAR(v_difference_percent, 'FM999999990.00'),
            TO_CHAR(v_threshold_percent, 'FM999999990.0');
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_check_price_difference_valekzhanin ON sale;
```

```

CREATE TRIGGER trg_check_price_difference_valekzhanin
BEFORE INSERT OR UPDATE ON sale
FOR EACH ROW
EXECUTE FUNCTION check_price_difference_valekzhanin();

```

```

real_estate_db=# \sf check_price_difference_valekzhanin
CREATE OR REPLACE FUNCTION public.check_price_difference_valekzhanin()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
    v_listed_price NUMERIC;
    v_sale_price NUMERIC;
    v_difference_percent NUMERIC;
    v_threshold_percent CONSTANT NUMERIC := 20.0;
BEGIN
    SELECT price INTO v_listed_price
    FROM property
    WHERE id = NEW.property_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;
    END IF;

    IF v_listed_price IS NULL OR NEW.price IS NULL THEN
        RAISE EXCEPTION 'Цена не может быть NULL';
    END IF;

    IF v_listed_price <= 0 OR NEW.price <= 0 THEN
        RAISE EXCEPTION 'Цена должна быть положительным числом';
    END IF;

    v_sale_price := NEW.price;
    v_difference_percent := ABS((v_sale_price - v_listed_price) / v_listed_price * 100);

    IF v_difference_percent > v_threshold_percent THEN
        RAISE WARNING 'ВНИМАНИЕ: Разница между заявленной (%) и продажной (%) стоимостью составляет % (порог: % )',
            TO_CHAR(v_listed_price, 'FM999999990.00'),
            TO_CHAR(v_sale_price, 'FM999999990.00'),
            TO_CHAR(v_difference_percent, 'FM999999990.00'),
            TO_CHAR(v_threshold_percent, 'FM999999990.0');
    END IF;

    RETURN NEW;
END;
$function$

```

Тесты

```

ALTER TABLE sale DISABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 1: Разница больше 20 процентов';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (3, CURRENT_DATE, 1, 750000);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
BEGIN;
DO $$

```

```
BEGIN
    RAISE NOTICE 'Тест 2: Не существующий объект';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (999999, CURRENT_DATE, 1, 500000);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 3: Отрицательная цена';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (7, CURRENT_DATE, 1, -500000);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 4: Разница меньше 20 процентов';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (3, CURRENT_DATE, 1, 450000);
    RAISE NOTICE 'Тест 4 пройден: предупреждение НЕ появилось';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 4 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
ALTER TABLE sale ENABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;
```

Вывод тестов

```
|real_estate_db=# \i 1.sql
ALTER TABLE
BEGIN
psql:1.sql:11: NOTICE:  Тест 1: Разница больше 20 процентов
psql:1.sql:11: WARNING:  ВНИМАНИЕ: Разница между заявленной (400253.00) и продажной (750000.00) стоимостью составляет 87.38 (порог: 20.0 )
psql:1.sql:11: NOTICE:  Journal: INSERT operation on sale
psql:1.sql:11: NOTICE:  Бонус обновлен для риэлтора 1
DO
ROLLBACK
BEGIN
psql:1.sql:22: NOTICE:  Тест 2: Не существующий объект
psql:1.sql:22: NOTICE:  Тест 2 пройден: ошибка корректно перехвачена: Объект недвижимости с ID 999999 не найден
DO
ROLLBACK
BEGIN
psql:1.sql:33: NOTICE:  Тест 3: Отрицательная цена
psql:1.sql:33: NOTICE:  Тест 3 пройден: ошибка корректно перехвачена: Цена должна быть положительным числом
DO
ROLLBACK
BEGIN
psql:1.sql:45: NOTICE:  Тест 4: Разница меньше 20 процентов
psql:1.sql:45: NOTICE:  Journal: INSERT operation on sale
psql:1.sql:45: NOTICE:  Бонус обновлен для риэлтора 1
psql:1.sql:45: NOTICE:  Тест 4 пройден: предупреждение НЕ появилось
DO
ROLLBACK
ALTER TABLE
```

Задание 2

2. Создать триггер, который при добавлении новой продажи будет осуществлять проверку на повторную продажу объекта недвижимости. Если в таблице «Продажи» уже имеется запись о продаже данного объекта, выводить соответствующее сообщение и запрещать новую продажу.

```
CREATE OR REPLACE FUNCTION prevent_duplicate_sale_valekzhanin()  
RETURNS TRIGGER AS $$
```

```
DECLARE
```

```
    v_existing_sale RECORD;
```

```
BEGIN
```

```
    SELECT * INTO v_existing_sale
```

```
    FROM sale
```

```
    WHERE property_id = NEW.property_id;
```

```
    IF FOUND THEN
```

```
        RAISE EXCEPTION 'Объект недвижимости с ID % уже был продан. Повторная  
продажа запрещена.',
```

```
        NEW.property_id;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_prevent_duplicate_sale_valekzhanin ON sale;
```

```
CREATE TRIGGER trg_prevent_duplicate_sale_valekzhanin
```

```
BEFORE INSERT ON sale
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION prevent_duplicate_sale_valekzhanin();
```

```
function  
real_estate_db=# \sf prevent_duplicate_sale_valekzhanin  
CREATE OR REPLACE FUNCTION public.prevent_duplicate_sale_valekzhanin()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $function$  
DECLARE  
    v_existing_sale RECORD;  
BEGIN  
    SELECT * INTO v_existing_sale  
    FROM sale  
    WHERE property_id = NEW.property_id;  
  
    IF FOUND THEN  
        RAISE EXCEPTION 'Объект недвижимости с ID % уже был продан. Повторная продажа запрещена.',  
        NEW.property_id;  
    END IF;  
  
    RETURN NEW;  
END;  
$function$
```

Тесты

```
BEGIN;  
DO $$  
BEGIN  
    RAISE NOTICE 'Тест 1: Повторная продажа объекта';  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (3, CURRENT_DATE, 1, 1000000);  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (3, CURRENT_DATE, 2, 1200000);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;  
END $$;  
ROLLBACK;  
BEGIN;  
DO $$  
BEGIN  
    RAISE NOTICE 'Тест 2: Первая продажа объекта';  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (7, CURRENT_DATE, 1, 1000000);  
    RAISE NOTICE 'Тест 2 пройден: объект продан успешно';  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;  
END $$;  
ROLLBACK;  
BEGIN;  
DO $$  
BEGIN  
    RAISE NOTICE 'Тест 3: Повторная продажа того же объекта';  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (7, CURRENT_DATE, 1, 1000000);  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (7, CURRENT_DATE, 2, 1200000);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;  
END $$;  
ROLLBACK;
```


Вывод тестов

```
|real_estate_db=# \i 2.sql
BEGIN
psql:2.sql:12: NOTICE:   Тест 1: Повторная продажа объекта
psql:2.sql:12: WARNING:   ВНИМАНИЕ: Разница между заявленной (400253.00) и продажной (1000000.00) стоимостью составляет 149.84 (порог: 20.0 )
psql:2.sql:12: NOTICE:   Journal: INSERT operation on sale
psql:2.sql:12: NOTICE:   Бонус обновлен для риэлтора 1
psql:2.sql:12: WARNING:   ВНИМАНИЕ: Разница между заявленной (400253.00) и продажной (1200000.00) стоимостью составляет 199.81 (порог: 20.0 )
psql:2.sql:12: NOTICE:   Тест 1 пройден: ошибка корректно перехвачена: Объект недвижимости с ID 3 уже был продан. Повторная продажа запрещена.
DO
ROLLBACK
BEGIN
psql:2.sql:24: NOTICE:   Тест 2: Первая продажа объекта
psql:2.sql:24: WARNING:   ВНИМАНИЕ: Разница между заявленной (4320000.00) и продажной (1000000.00) стоимостью составляет 76.85 (порог: 20.0 )
psql:2.sql:24: NOTICE:   Journal: INSERT operation on sale
psql:2.sql:24: NOTICE:   Бонус обновлен для риэлтора 1
psql:2.sql:24: NOTICE:   Тест 2 пройден: объект продан успешно
DO
ROLLBACK
BEGIN
psql:2.sql:37: NOTICE:   Тест 3: Повторная продажа того же объекта
psql:2.sql:37: WARNING:   ВНИМАНИЕ: Разница между заявленной (4320000.00) и продажной (1000000.00) стоимостью составляет 76.85 (порог: 20.0 )
psql:2.sql:37: NOTICE:   Journal: INSERT operation on sale
psql:2.sql:37: NOTICE:   Бонус обновлен для риэлтора 1
psql:2.sql:37: WARNING:   ВНИМАНИЕ: Разница между заявленной (4320000.00) и продажной (1200000.00) стоимостью составляет 72.22 (порог: 20.0 )
psql:2.sql:37: NOTICE:   Тест 3 пройден: ошибка корректно перехвачена: Объект недвижимости с ID 7 уже был продан. Повторная продажа запрещена.
DO
ROLLBACK
```

Задание 3

3. Создать триггер, который при добавлении новой записи в таблицу «Структура объекта недвижимости» проверяет несоответствие суммы площадей всех заявленных комнат (в большую сторону) с общей площадью объекта недвижимости. Выводить сообщение на сколько превышена площадь.

```
DROP TABLE IF EXISTS property_structure;
```

```
CREATE TABLE property_structure (  
    id SERIAL PRIMARY KEY,  
    property_id INTEGER REFERENCES property(id),  
    room_area NUMERIC NOT NULL  
);
```

```
INSERT INTO property_structure (property_id, room_area) VALUES  
(1, 10),  
(1, 15),  
(2, 20),  
(2, 25),  
(3, 12);
```

```
CREATE OR REPLACE FUNCTION check_structure_area_valekzhanin()  
RETURNS TRIGGER AS $$  
DECLARE
```

```
    v_property_area NUMERIC;  
    v_current_sum NUMERIC;  
    v_total_sum NUMERIC;  
    v_excess NUMERIC;
```

```
BEGIN
```

```
    SELECT area INTO v_property_area  
    FROM property  
    WHERE id = NEW.property_id;
```

```
    IF NOT FOUND THEN
```

```
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;  
    END IF;
```

```
    SELECT COALESCE(SUM(room_area), 0) INTO v_current_sum  
    FROM property_structure  
    WHERE property_id = NEW.property_id;
```

```
    v_total_sum := v_current_sum + NEW.room_area;
```

```
    IF v_total_sum > v_property_area THEN
```

```
        v_excess := v_total_sum - v_property_area;
```

```
        RAISE NOTICE 'ВНИМАНИЕ: Превышение площади составляет % кв.м. (Сумма: %, Лимит: %)',
```

```

        TO_CHAR(v_excess, 'FM999999990.00'),
        TO_CHAR(v_total_sum, 'FM999999990.00'),
        TO_CHAR(v_property_area, 'FM999999990.00');
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

DROP TRIGGER IF EXISTS trg_check_structure_area_valekzhanin ON property_structure;

```

```

CREATE TRIGGER trg_check_structure_area_valekzhanin
BEFORE INSERT ON property_structure
FOR EACH ROW
EXECUTE FUNCTION check_structure_area_valekzhanin();

```

```

real_estate_db=# \sf check_structure_area_valekzhanin
CREATE OR REPLACE FUNCTION public.check_structure_area_valekzhanin()
    RETURNS trigger
    LANGUAGE plpgsql
    AS $function$
DECLARE
    v_property_area NUMERIC;
    v_current_sum NUMERIC;
    v_total_sum NUMERIC;
    v_excess NUMERIC;
BEGIN
    SELECT area INTO v_property_area
    FROM property
    WHERE id = NEW.property_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;
    END IF;

    SELECT COALESCE(SUM(room_area), 0) INTO v_current_sum
    FROM property_structure
    WHERE property_id = NEW.property_id;

    v_total_sum := v_current_sum + NEW.room_area;

    IF v_total_sum > v_property_area THEN
        v_excess := v_total_sum - v_property_area;
        RAISE NOTICE 'ВНИМАНИЕ: Превышение площади составляет % кв.м. (Сумма: %, Лимит: %)',
            TO_CHAR(v_excess, 'FM999999990.00'),
            TO_CHAR(v_total_sum, 'FM999999990.00'),
            TO_CHAR(v_property_area, 'FM999999990.00');
    END IF;

    RETURN NEW;
END;
$function$

```

Тесты

```

BEGIN;
DO $$

```

```
BEGIN
    RAISE NOTICE 'Тест 1: Сумма площадей в пределах нормы';
    INSERT INTO property_structure (property_id, room_area)
    VALUES (14, 20);
    RAISE NOTICE 'Тест 1 пройден: предупреждение НЕ появилось';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
```

```
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 2: Превышение общей площади объекта';
    INSERT INTO property_structure (property_id, room_area)
    VALUES (14, 100);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
```

```
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 3: Накопительное превышение площади';
    INSERT INTO property_structure (property_id, room_area)
    VALUES (18, 50);
    INSERT INTO property_structure (property_id, room_area)
    VALUES (18, 50);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
```

Вывод тестов

```
real_estate_db=# \i 3.sql
BEGIN
psql:3.sql:11: NOTICE:  Тест 1: Сумма площадей в пределах нормы
psql:3.sql:11: NOTICE:  Тест 1 пройден: предупреждение НЕ появилось
DO
ROLLBACK
BEGIN
psql:3.sql:23: NOTICE:  Тест 2: Превышение общей площади объекта
psql:3.sql:23: NOTICE:  ВНИМАНИЕ: Превышение площади составляет 67.00 кв.м. (Сумма: 100.00, Лимит: 33.00)
DO
ROLLBACK
BEGIN
psql:3.sql:37: NOTICE:  Тест 3: Накопительное превышение площади
psql:3.sql:37: NOTICE:  ВНИМАНИЕ: Превышение площади составляет 3.00 кв.м. (Сумма: 50.00, Лимит: 47.00)
psql:3.sql:37: NOTICE:  ВНИМАНИЕ: Превышение площади составляет 53.00 кв.м. (Сумма: 100.00, Лимит: 47.00)
DO
ROLLBACK
```

Задание 4

4. Создать триггер, который будет выводить предупреждающее сообщение, что дата продажи раньше, чем дата размещения объявления.

```
CREATE OR REPLACE FUNCTION check_sale_date_valekzhanin()
RETURNS TRIGGER AS $$
DECLARE
    v_listing_date DATE;
BEGIN
    SELECT listing_date INTO v_listing_date
    FROM property
    WHERE id = NEW.property_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;
    END IF;

    IF NEW.sale_date < v_listing_date THEN
        RAISE NOTICE 'ВНИМАНИЕ: Дата продажи (%) раньше даты размещения
объявления (%)',
            NEW.sale_date, v_listing_date;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_check_sale_date_valekzhanin ON sale;

CREATE TRIGGER trg_check_sale_date_valekzhanin
BEFORE INSERT OR UPDATE ON sale
FOR EACH ROW
EXECUTE FUNCTION check_sale_date_valekzhanin();
```

```

real_estate_db=# \sf check_sale_date_valekzhanin
CREATE OR REPLACE FUNCTION public.check_sale_date_valekzhanin()
  RETURNS trigger
  LANGUAGE plpgsql
AS $function$
DECLARE
    v_listing_date DATE;
BEGIN
    SELECT listing_date INTO v_listing_date
    FROM property
    WHERE id = NEW.property_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Объект недвижимости с ID % не найден', NEW.property_id;
    END IF;

    IF NEW.sale_date < v_listing_date THEN
        RAISE NOTICE 'ВНИМАНИЕ: Дата продажи (%) раньше даты размещения объявления (%)',
            NEW.sale_date, v_listing_date;
    END IF;

    RETURN NEW;
END;
$function$

```

Тесты

```

ALTER TABLE sale DISABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 1: Дата продажи корректна';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (21, '2025-09-01', 1, 5000000);
    RAISE NOTICE 'Тест 1 пройден: предупреждение НЕ появилось';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 2: Дата продажи раньше размещения';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (23, '2022-01-01', 1, 5000000);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

```
ALTER TABLE sale ENABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;
```

Вывод тестов

```
|real_estate_db=# \i 4.sql
ALTER TABLE
BEGIN
psql:4.sql:12: NOTICE:  Тест 1: Дата продажи корректна
psql:4.sql:12: WARNING:  ВНИМАНИЕ: Разница между заявленной (165502.00) и продажной (5000000.00) стоимостью составляет 2921.11 (порог: 20.0 )
psql:4.sql:12: NOTICE:  Journal: INSERT operation on sale
psql:4.sql:12: NOTICE:  Бонус обновлен для риэлтора 1
psql:4.sql:12: NOTICE:  Тест 1 пройден: предупреждение НЕ появилось
DO
ROLLBACK
BEGIN
psql:4.sql:23: NOTICE:  Тест 2: Дата продажи раньше размещения
psql:4.sql:23: WARNING:  ВНИМАНИЕ: Разница между заявленной (241357.00) и продажной (5000000.00) стоимостью составляет 1971.62 (порог: 20.0 )
psql:4.sql:23: NOTICE:  ВНИМАНИЕ: Дата продажи (2022-01-01) раньше даты размещения объявления (2025-08-09)
psql:4.sql:23: NOTICE:  Journal: INSERT operation on sale
psql:4.sql:23: NOTICE:  Бонус обновлен для риэлтора 1
DO
ROLLBACK
ALTER TABLE
```


Задание 5

5. Создать таблицы «Журнал»: дата и время операции, операция (INSERT, UPDATE, DELETE), пользователь. Создать триггер, который будет формировать новую запись в этой таблице при добавлении/изменении/удалении строки в таблицу «Продажи».

```
DROP TABLE IF EXISTS journal;
```

```
CREATE TABLE journal (  
    id SERIAL PRIMARY KEY,  
    datetime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    action VARCHAR(10) NOT NULL,  
    user_name VARCHAR(255) NOT NULL  
);
```

```
CREATE OR REPLACE FUNCTION log_sale_action_valekzhanin()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO journal (datetime, action, user_name)  
    VALUES (CURRENT_TIMESTAMP, TG_OP, current_user);  
  
    IF TG_OP = 'INSERT' THEN  
        RAISE NOTICE 'Journal: INSERT operation on sale';  
    ELSIF TG_OP = 'UPDATE' THEN  
        RAISE NOTICE 'Journal: UPDATE operation on sale';  
    ELSIF TG_OP = 'DELETE' THEN  
        RAISE NOTICE 'Journal: DELETE operation on sale';  
    END IF;  
  
    RETURN NULL;  
END;  
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_log_sale_action_valekzhanin ON sale;
```

```
CREATE TRIGGER trg_log_sale_action_valekzhanin  
AFTER INSERT OR UPDATE OR DELETE ON sale  
FOR EACH ROW  
EXECUTE FUNCTION log_sale_action_valekzhanin();
```

```

real_estate_db=# \sf log_sale_action_valekzhanin
CREATE OR REPLACE FUNCTION public.log_sale_action_valekzhanin()
  RETURNS trigger
  LANGUAGE plpgsql
AS $function$
BEGIN
    INSERT INTO journal (datetime, action, user_name)
    VALUES (CURRENT_TIMESTAMP, TG_OP, current_user);

    IF TG_OP = 'INSERT' THEN
        RAISE NOTICE 'Journal: INSERT operation on sale';
    ELSIF TG_OP = 'UPDATE' THEN
        RAISE NOTICE 'Journal: UPDATE operation on sale';
    ELSIF TG_OP = 'DELETE' THEN
        RAISE NOTICE 'Journal: DELETE operation on sale';
    END IF;

    RETURN NULL;
END;
$function$

```

Тесты

```
ALTER TABLE sale DISABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;
```

```

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 1: Проверка INSERT';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (26, CURRENT_DATE, 1, 1000000);
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

```

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 2: Проверка UPDATE';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (28, CURRENT_DATE, 1, 2000000);
    UPDATE sale SET price = 2500000
    WHERE property_id = 28 AND realtor_id = 1;

```

```

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
DECLARE
    v_sale_id INTEGER;
BEGIN
    RAISE NOTICE 'Тест 3: Проверка DELETE';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (30, CURRENT_DATE, 1, 3000000)
    RETURNING id INTO v_sale_id;
    DELETE FROM sale WHERE id = v_sale_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

ALTER TABLE sale ENABLE TRIGGER trg_prevent_duplicate_sale_valekzhanin;

```

Вывод тестов

```

|real_estate_db=# \i 5.sql
ALTER TABLE
BEGIN
psql:5.sql:12: NOTICE:  Тест 1: Проверка INSERT
psql:5.sql:12: WARNING:  ВНИМАНИЕ: Разница между заявленной (252888.00) и продажной (1000000.00) стоимостью составляет 295.43 (порог: 20.0 )
psql:5.sql:12: NOTICE:  Journal: INSERT operation on sale
psql:5.sql:12: NOTICE:  Бонус обновлен для риэлтора 1
DO
ROLLBACK
BEGIN
psql:5.sql:26: NOTICE:  Тест 2: Проверка UPDATE
psql:5.sql:26: WARNING:  ВНИМАНИЕ: Разница между заявленной (296869.00) и продажной (2000000.00) стоимостью составляет 573.70 (порог: 20.0 )
psql:5.sql:26: NOTICE:  Journal: INSERT operation on sale
psql:5.sql:26: NOTICE:  Бонус обновлен для риэлтора 1
psql:5.sql:26: WARNING:  ВНИМАНИЕ: Разница между заявленной (296869.00) и продажной (2500000.00) стоимостью составляет 742.12 (порог: 20.0 )
psql:5.sql:26: NOTICE:  Journal: UPDATE operation on sale
DO
ROLLBACK
BEGIN
psql:5.sql:42: NOTICE:  Тест 3: Проверка DELETE
psql:5.sql:42: WARNING:  ВНИМАНИЕ: Разница между заявленной (170424.00) и продажной (3000000.00) стоимостью составляет 1660.32 (порог: 20.0 )
psql:5.sql:42: NOTICE:  Journal: INSERT operation on sale
psql:5.sql:42: NOTICE:  Бонус обновлен для риэлтора 1
psql:5.sql:42: NOTICE:  Journal: DELETE operation on sale
psql:5.sql:42: NOTICE:  Бонус уменьшен для риэлтора 1
DO
ROLLBACK
ALTER TABLE

```

Работа с триггерами. Часть 2

Цель: изучить работу с триггерами.

Вывод \df

List of functions				
Schema	Name	Result data type	Argument data types	Type
public	check_bonus_limit_valekzhanin	trigger		func
public	check_passport_format_valekzhanin	trigger		func
public	check_price_difference_valekzhanin	trigger		func
public	check_sale_date_valekzhanin	trigger		func
public	check_structure_area_valekzhanin	trigger		func
public	format_phone_valekzhanin	trigger		func
public	log_sale_action_valekzhanin	trigger		func
public	prevent_duplicate_sale_valekzhanin	trigger		func
public	update_realtor_bonus_valekzhanin	trigger		func
(9 rows)				

Задание 1

1. Добавить таблицу «Бонусы», в которой будет две колонки: код риэлтора и размер накопленных бонусов. Размер бонуса рассчитывается по формуле – стоимость продажи*5%. Создать триггер, который будет автоматически увеличивать размер накопленного бонуса при добавлении новой продажи. Необходимо учесть, что продажа риэлтора может быть впервые, следовательно необходимо добавить новую запись в таблицу. При удалении данных о продаже, размер накопленного бонуса должен уменьшиться.

```
DROP TABLE IF EXISTS bonus;
```

```
CREATE TABLE bonus (  
    realtor_id INTEGER PRIMARY KEY REFERENCES realtor(id),  
    amount NUMERIC(15, 2) NOT NULL DEFAULT 0  
);
```

```
INSERT INTO bonus (realtor_id, amount)  
SELECT id, 0 FROM realtor;
```

```
CREATE OR REPLACE FUNCTION update_realtor_bonus_valekzhanin()  
RETURNS TRIGGER AS $$  
DECLARE  
    v_exists BOOLEAN;  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        SELECT EXISTS(SELECT 1 FROM bonus WHERE realtor_id = NEW.realtor_id) INTO  
v_exists;  
        IF v_exists THEN  
            UPDATE bonus SET amount = amount + NEW.price * 0.05 WHERE realtor_id =  
NEW.realtor_id;  
            RAISE NOTICE 'Бонус обновлен для риэлтора %', NEW.realtor_id;  
        ELSE  
            INSERT INTO bonus (realtor_id, amount) VALUES (NEW.realtor_id, NEW.price *  
0.05);  
            RAISE NOTICE 'Запись бонуса создана для риэлтора %', NEW.realtor_id;  
        END IF;  
        RETURN NEW;  
    ELSIF TG_OP = 'DELETE' THEN  
        UPDATE bonus SET amount = amount - OLD.price * 0.05 WHERE realtor_id =  
OLD.realtor_id;  
        RAISE NOTICE 'Бонус уменьшен для риэлтора %', OLD.realtor_id;  
        RETURN OLD;  
    END IF;  
    RETURN NULL;  
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_update_realtor_bonus_valekzhanin ON sale;  
CREATE TRIGGER trg_update_realtor_bonus_valekzhanin  
AFTER INSERT OR DELETE ON sale  
FOR EACH ROW EXECUTE FUNCTION update_realtor_bonus_valekzhanin();
```

```
real_estate_db=# \sf update_realtor_bonus_valekzhanin  
CREATE OR REPLACE FUNCTION public.update_realtor_bonus_valekzhanin()  
    RETURNS trigger  
    LANGUAGE plpgsql  
AS $function$  
DECLARE  
    v_exists BOOLEAN;  
BEGIN  
    IF TG_OP = 'INSERT' THEN  
        SELECT EXISTS(SELECT 1 FROM bonus WHERE realtor_id = NEW.realtor_id) INTO v_exists;  
        IF v_exists THEN  
            UPDATE bonus SET amount = amount + NEW.price * 0.05 WHERE realtor_id = NEW.realtor_id;  
            RAISE NOTICE 'Бонус обновлен для риэлтора %', NEW.realtor_id;  
        ELSE  
            INSERT INTO bonus (realtor_id, amount) VALUES (NEW.realtor_id, NEW.price * 0.05);  
            RAISE NOTICE 'Запись бонуса создана для риэлтора %', NEW.realtor_id;  
        END IF;  
        RETURN NEW;  
    ELSIF TG_OP = 'DELETE' THEN  
        UPDATE bonus SET amount = amount - OLD.price * 0.05 WHERE realtor_id = OLD.realtor_id;  
        RAISE NOTICE 'Бонус уменьшен для риэлтора %', OLD.realtor_id;  
        RETURN OLD;  
    END IF;  
    RETURN NULL;  
END;  
$function$
```

Тесты

```
BEGIN;  
DO $$  
BEGIN  
    RAISE NOTICE 'Тест 1: Обновление существующего бонуса';  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (3, CURRENT_DATE, 1, 1000000);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;  
END $$;  
ROLLBACK;
```

```
BEGIN;  
DO $$  
BEGIN  
    RAISE NOTICE 'Тест 2: Создание новой записи бонуса';  
    DELETE FROM bonus WHERE realtor_id = 2;  
    INSERT INTO sale (property_id, sale_date, realtor_id, price)  
    VALUES (7, CURRENT_DATE, 2, 2000000);
```

```

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
DECLARE
    v_sale_id INTEGER;
BEGIN
    RAISE NOTICE 'Тест 3: Уменьшение бонуса при удалении продажи';
    INSERT INTO sale (property_id, sale_date, realtor_id, price)
    VALUES (11, CURRENT_DATE, 3, 3000000)
    RETURNING id INTO v_sale_id;

    DELETE FROM sale WHERE id = v_sale_id;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

Вывод тестов

```

|real_estate_db=# \i 1.sql
BEGIN
psql:1.sql:10: NOTICE:  Тест 1: Обновление существующего бонуса
psql:1.sql:10: WARNING:  ВНИМАНИЕ: Разница между заявленной (400253.00) и продажной (1000000.00) стоимостью составляет 149.84 (порог: 20.0 )
psql:1.sql:10: NOTICE:  Journal: INSERT operation on sale
psql:1.sql:10: NOTICE:  Бонус обновлен для риэлтора 1
DO
ROLLBACK
BEGIN
psql:1.sql:23: NOTICE:  Тест 2: Создание новой записи бонуса
psql:1.sql:23: WARNING:  ВНИМАНИЕ: Разница между заявленной (4320000.00) и продажной (2000000.00) стоимостью составляет 53.70 (порог: 20.0 )
psql:1.sql:23: NOTICE:  Journal: INSERT operation on sale
psql:1.sql:23: NOTICE:  Запись бонуса создана для риэлтора 2
DO
ROLLBACK
BEGIN
psql:1.sql:40: NOTICE:  Тест 3: Уменьшение бонуса при удалении продажи
psql:1.sql:40: WARNING:  ВНИМАНИЕ: Разница между заявленной (11173.00) и продажной (3000000.00) стоимостью составляет 26750.44 (порог: 20.0 )
psql:1.sql:40: NOTICE:  Journal: INSERT operation on sale
psql:1.sql:40: NOTICE:  Бонус обновлен для риэлтора 3
psql:1.sql:40: NOTICE:  Journal: DELETE operation on sale
psql:1.sql:40: NOTICE:  Бонус уменьшен для риэлтора 3
DO
ROLLBACK

```

Задание 2

2. Создать триггер, который будет информировать о превышении размера накопленного бонуса риэлтором. Максимальный размер бонуса установить самостоятельно.

```
DROP TABLE IF EXISTS bonus;
```

```
CREATE TABLE bonus (  
    realtor_id INTEGER PRIMARY KEY REFERENCES realtor(id),  
    amount NUMERIC(15, 2) NOT NULL DEFAULT 0  
);
```

```
INSERT INTO bonus (realtor_id, amount)  
SELECT id, 0 FROM realtor;
```

```
CREATE OR REPLACE FUNCTION check_bonus_limit_valekzhanin()  
RETURNS TRIGGER AS $$
```

```
DECLARE
```

```
    v_max_bonus CONSTANT NUMERIC := 500000;
```

```
    v_current_bonus NUMERIC;
```

```
BEGIN
```

```
    SELECT COALESCE(amount, 0) INTO v_current_bonus  
    FROM bonus
```

```
    WHERE realtor_id = NEW.realtor_id;
```

```
    IF TG_OP = 'INSERT' THEN
```

```
        v_current_bonus := v_current_bonus + NEW.amount;
```

```
    ELSIF TG_OP = 'UPDATE' THEN
```

```
        v_current_bonus := NEW.amount;
```

```
    END IF;
```

```
    IF v_current_bonus > v_max_bonus THEN
```

```
        RAISE NOTICE 'ВНИМАНИЕ: Накопленный бонус риэлтора % (%) превысил лимит  
(%)',
```

```
        NEW.realtor_id,
```

```
        TO_CHAR(v_current_bonus, 'FM999999999.00'),
```

```
        TO_CHAR(v_max_bonus, 'FM999999999.00');
```

```
    END IF;
```

```
    RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_check_bonus_limit_valekzhanin ON bonus;
```

```
CREATE TRIGGER trg_check_bonus_limit_valekzhanin  
AFTER INSERT OR UPDATE ON bonus
```


FOR EACH ROW

EXECUTE FUNCTION check_bonus_limit_valekzhanin();

```
real_estate_db=# \sf check_bonus_limit_valekzhanin
CREATE OR REPLACE FUNCTION public.check_bonus_limit_valekzhanin()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
    v_max_bonus CONSTANT NUMERIC := 500000;
    v_current_bonus NUMERIC;
BEGIN
    SELECT COALESCE(amount, 0) INTO v_current_bonus
    FROM bonus
    WHERE realtor_id = NEW.realtor_id;

    IF TG_OP = 'INSERT' THEN
        v_current_bonus := v_current_bonus + NEW.amount;
    ELSIF TG_OP = 'UPDATE' THEN
        v_current_bonus := NEW.amount;
    END IF;

    IF v_current_bonus > v_max_bonus THEN
        RAISE NOTICE 'ВНИМАНИЕ: Накопленный бонус риэлтора % (%) превысил лимит (%)',
            NEW.realtor_id,
            TO_CHAR(v_current_bonus, 'FM999999999.00'),
            TO_CHAR(v_max_bonus, 'FM999999999.00');
    END IF;

    RETURN NULL;
END;
$function$
```

Тесты

BEGIN;

DO \$\$

BEGIN

RAISE NOTICE 'Тест 1: Бонус в пределах нормы';

UPDATE bonus SET amount = 100000 WHERE realtor_id = 1;

EXCEPTION

WHEN OTHERS THEN

RAISE NOTICE 'Ошибка в тесте 1: %', SQLERRM;

END \$\$;

ROLLBACK;

BEGIN;

DO \$\$

BEGIN

RAISE NOTICE 'Тест 2: Превышение лимита бонуса';

UPDATE bonus SET amount = 600000 WHERE realtor_id = 2;

EXCEPTION

```
    WHEN OTHERS THEN
        RAISE NOTICE 'Ошибка в тесте 2: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 3: Превышение лимита при обновлении бонуса';
    UPDATE bonus SET amount = 600000 WHERE realtor_id = 3;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Ошибка в тесте 3: %', SQLERRM;
END $$;
ROLLBACK;
```

Вывод тестов

```
[real_estate_db=# \i 2.sql
BEGIN
psql:2.sql:9: NOTICE:  Тест 1: Бонус в пределах нормы
DO
ROLLBACK
BEGIN
psql:2.sql:20: NOTICE:  Тест 2: Превышение лимита бонуса
DO
ROLLBACK
BEGIN
psql:2.sql:31: NOTICE:  Тест 3: Превышение лимита при обновлении бонуса
DO
ROLLBACK
```

Задание 3

3. В таблицу «Риэлторы» добавить колонку «Паспортные данные». Создать триггер, который будет проверять корректность паспортных данных по маске: XXXX УУУУУУ, где X – серия паспорта, У – номер паспорта. Между серией и номером паспорта пробел. При несоответствии вводимой информации маске, выводить сообщение.

```
ALTER TABLE realtor ADD COLUMN IF NOT EXISTS passport_data VARCHAR(11);
```

```
CREATE OR REPLACE FUNCTION check_passport_format_valekzhanin()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF NEW.passport_data IS NOT NULL THEN
```

```
        IF NEW.passport_data !~ '^\d{4} \d{6}$' THEN
```

```
            RAISE EXCEPTION 'Некорректные паспортные данные. Ожидается формат: XXXX  
УУУУУУ (получено: %)', NEW.passport_data;
```

```
        END IF;
```

```
    END IF;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_check_passport_format_valekzhanin ON realtor;
```

```
CREATE TRIGGER trg_check_passport_format_valekzhanin
```

```
BEFORE INSERT OR UPDATE ON realtor
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION check_passport_format_valekzhanin();
```

```
real_estate_db=# \sf check_passport_format_valekzhanin
CREATE OR REPLACE FUNCTION public.check_passport_format_valekzhanin()
    RETURNS trigger
    LANGUAGE plpgsql
    AS $function$
BEGIN
    IF NEW.passport_data IS NOT NULL THEN
        IF NEW.passport_data !~ '^\d{4} \d{6}$' THEN
            RAISE EXCEPTION 'Некорректные паспортные данные. Ожидается формат: XXXX УУУУУУ (получено: %)', NEW.passport_data;
        END IF;
    END IF;

    RETURN NEW;
END;
$function$
```

Тесты

```
BEGIN;
```

```
DO $$
```

```
BEGIN
```

```
    RAISE NOTICE 'Тест 1: Корректные паспортные данные';
```

```
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number, passport_data)
```

```
    VALUES ('Тестов', 'Тест', 'Тестович', '+700000000000', '1234 567890');
```

```

    RAISE NOTICE 'Тест 1 пройден: запись добавлена';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 2: Некорректные паспортные данные (нарушение маски)';
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number, passport_data)
    VALUES ('Тестов', 'Тест', 'Тестович', '+700000000000', '123 4567890');
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 3: Некорректные паспортные данные (буквы вместо цифр)';
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number, passport_data)
    VALUES ('Тестов', 'Тест', 'Тестович', '+700000000000', '1234 ABCD90');
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 4: NULL значение паспортных данных (допустимо)';
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number, passport_data)
    VALUES ('Тестов', 'Тест', 'Тестович', '+700000000000', NULL);
    RAISE NOTICE 'Тест 4 пройден: запись добавлена';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 4 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

Вывод тестов

```
real_estate_db=# \i 3.sql
BEGIN
psql:3.sql:11: NOTICE:  Тест 1: Корректные паспортные данные
psql:3.sql:11: NOTICE:  Тест 1 пройден: запись добавлена
DO
ROLLBACK
BEGIN
psql:3.sql:23: NOTICE:  Тест 2: Некорректные паспортные данные (нарушение маски)
psql:3.sql:23: NOTICE:  Тест 2 пройден: ошибка корректно перехвачена: Некорректные паспортные данные. Ожидается формат: XXXX YYYYYY (получено: 123 4567890)
DO
ROLLBACK
BEGIN
psql:3.sql:35: NOTICE:  Тест 3: Некорректные паспортные данные (буквы вместо цифр)
psql:3.sql:35: NOTICE:  Тест 3 пройден: ошибка корректно перехвачена: Некорректные паспортные данные. Ожидается формат: XXXX YYYYYY (получено: 1234 ABCD90)
DO
ROLLBACK
BEGIN
psql:3.sql:48: NOTICE:  Тест 4: NULL значение паспортных данных (допустимо)
psql:3.sql:48: NOTICE:  Тест 4 пройден: запись добавлена
DO
ROLLBACK
```

Задание 4

4. Создать триггер, который при добавлении нового риэлтора формат телефона:

79996667788 преобразует в: +7 (999) 666 77 88.

```
CREATE OR REPLACE FUNCTION format_phone_valekzhanin()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.phone_number ~ '^\\d{11}$' THEN
        NEW.phone_number := regexp_replace(NEW.phone_number,
        '\\d\\d{3}\\d{3}\\d{2}\\d{2}', '+\\1 (\\2) \\3 \\4 \\5');
        RAISE NOTICE 'Телефон риэлтора отформатирован: %', NEW.phone_number;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
DROP TRIGGER IF EXISTS trg_format_phone_valekzhanin ON reator;
```

```
CREATE TRIGGER trg_format_phone_valekzhanin
BEFORE INSERT OR UPDATE ON reator
FOR EACH ROW
EXECUTE FUNCTION format_phone_valekzhanin();
```

```
real_estate_db=# \\sf format_phone_valekzhanin
CREATE OR REPLACE FUNCTION public.format_phone_valekzhanin()
    RETURNS trigger
    LANGUAGE plpgsql
    AS $function$
BEGIN
    IF NEW.phone_number ~ '^\\d{11}$' THEN
        NEW.phone_number := regexp_replace(NEW.phone_number, '\\d\\d{3}\\d{3}\\d{2}\\d{2}', '+\\1 (\\2) \\3 \\4 \\5');
        RAISE NOTICE 'Телефон риэлтора отформатирован: %', NEW.phone_number;
    END IF;

    RETURN NEW;
END;
$function$
```

Тесты

```
BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 1: _raw_номер_требует_форматирования';
    INSERT INTO reator (last_name, first_name, middle_name, phone_number)
    VALUES ('Тестов', 'Тест', 'Тестович', '79996667788');
    RAISE NOTICE 'Тест 1 пройден: телефон отформатирован';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 1 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;
```

```

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 2: _номер_уже_отформатирован';
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number)
    VALUES ('Тестов', 'Тест', 'Тестович', '+7 (999) 666 77 88');
    RAISE NOTICE 'Тест 2 пройден: предупреждение не появилось';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 2 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

```

BEGIN;
DO $$
BEGIN
    RAISE NOTICE 'Тест 3: _некорректный_номер';
    INSERT INTO realtor (last_name, first_name, middle_name, phone_number)
    VALUES ('Тестов', 'Тест', 'Тестович', '12345');
    RAISE NOTICE 'Тест 3 пройден: форматирование не применено';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Тест 3 пройден: ошибка корректно перехвачена: %', SQLERRM;
END $$;
ROLLBACK;

```

Вывод тестов

```

[real_estate_db=# \i 4.sql
BEGIN
psql:4.sql:11: NOTICE:  Тест 1: _raw_номер_требует_форматирования
psql:4.sql:11: NOTICE:  Телефон риэлтора отформатирован: +7 (999) 666 77 88
psql:4.sql:11: NOTICE:  Тест 1 пройден: телефон отформатирован
DO
ROLLBACK
BEGIN
psql:4.sql:24: NOTICE:  Тест 2: _номер_уже_отформатирован
psql:4.sql:24: NOTICE:  Тест 2 пройден: предупреждение не появилось
DO
ROLLBACK
BEGIN
psql:4.sql:37: NOTICE:  Тест 3: _некорректный_номер
psql:4.sql:37: NOTICE:  Тест 3 пройден: форматирование не применено
DO
ROLLBACK

```